
Cahier des charges (DSF) pour la réalisation d'une Application Web

Gestion de membre d'une Association : AssoManager

1. Introduction

Ce document définit les spécifications fonctionnelles, techniques et les exigences générales pour le développement d'une application web intitulée "**AssoManager**" (nom provisoire).

L'objectif est de créer une **SPA React** connectée à une **API Laravel** afin de gérer une association avec un périmètre volontairement réduit pour s'entraîner au développement web.

Public cible :

- Administrateurs de l'association (gestion des membres + suivi des cotisations)
- Membres (compte, connexion, consultation de leurs paiements)

2. Objectifs du projet

Objectifs fonctionnels

- Mettre en place un système d'utilisateurs avec rôles (**admin**, **membre**).
- Gérer l'**authentification** (inscription membre, connexion, déconnexion).
- Permettre le **paiement et le suivi des cotisations mensuelles**.

Objectifs d'apprentissage (Laravel + React)

- Construire une API REST (Laravel) + une SPA (React).
- Implémenter l'auth (tokens) + gestion des rôles/permissions.
- Concevoir un modèle de données simple (migrations, relations).
- Pratiquer validation, erreurs, pagination, UI propre.

3. Portée du projet

Inclus dans la V1 (2 semaines)

- Authentification : inscription, connexion, déconnexion.
- Gestion utilisateurs : rôles (admin/membre), liste et détails (admin).
- Gestion membres : création/inscription, consultation du profil.
- Cotisations mensuelles : enregistrement d'un paiement, historique, statut "payé / non payé".

Exclu de la V1 (pour plus tard)

- Paiement en ligne (Orange Money / carte / Stripe).
- Gestion d'événements, réunions, documents, dons, dépenses.
- Relances automatiques par email/SMS.
- Multi-association / multi-tenant.
- Réinitialisation de mot de passe (optionnel, peut être ajouté si temps).

Utilisateurs types et droits

- **Admin**
 - Voir/chercher les membres
 - Gérer l'état d'un membre (actif/inactif)
 - Enregistrer les paiements de cotisations
 - Consulter l'historique global des paiements
- **Membre**
 - S'inscrire, se connecter, se déconnecter
 - Voir son profil
 - Voir ses paiements et son statut de cotisation

4. Exigences fonctionnelles

4.1 Authentification

Inscription (membre)

-
- Un visiteur peut créer un compte membre.
 - Champs minimum : nom, prénom, email (unique), mot de passe.
 - Validation : format email, mot de passe (min 8), champs obligatoires.

Connexion

- Un utilisateur peut se connecter avec email + mot de passe.
- Retour : token d'accès + informations utilisateur (id, nom, rôle).

Déconnexion

- Un utilisateur connecté peut se déconnecter.
- Invalidation du token (ou suppression côté client selon stratégie retenue).

4.2 Gestion des utilisateurs (admin)

Lister les utilisateurs / membres

- L'admin peut consulter la liste paginée des membres.
- Recherche : nom, prénom, email (au minimum).

Consulter un membre

- Détails : infos du profil + historique de paiements.
- Statut : actif / inactif.

Activer / Désactiver un membre

- Un membre inactif ne peut pas se connecter (ou accès limité : à décider).
- L'admin peut modifier ce statut.

4.3 Gestion des cotisations mensuelles

Règle métier

- Une cotisation correspond à **un mois** pour **un membre**.
- Un membre ne peut pas avoir 2 paiements "validés" pour le même mois/année.

Enregistrer un paiement (admin)

- L'admin peut ajouter un paiement pour un membre :
 - Mois (1–12), année
 - Montant
 - Date de paiement
 - Mode de paiement (cash / virement / autre)
 - Référence (optionnel)
- L'application calcule le statut : "payé" pour le mois concerné.

Consulter l'historique

- Admin : historique global (filtres par mois/année/membre).
- Membre : historique personnel.

État de cotisation

- Pour un mois donné, afficher : payé / non payé.
- Sur le tableau de bord membre : afficher le mois en cours.

5. Modèle de données / Structure des données

Entités principales

User

- id
- first_name

-
- last_name
 - email (unique)
 - password (hash)
 - role (enum: ADMIN, MEMBER) ou role_id
 - status (ACTIVE, INACTIVE)
 - created_at, updated_at

ContributionPayment (Paielement de cotisation)

- id
- user_id (FK -> User)
- year (ex: 2026)
- month (1–12)
- amount
- payment_date
- payment_method (CASH, TRANSFER, OTHER)
- reference (nullable)
- created_at, updated_at

Relations

- User (1) — (N) ContributionPayment
- Contrainte d'unicité : (user_id, year, month) unique

6. Exigences non fonctionnelles

Performance

- Liste des membres paginée (ex: 10/20 par page).
- Endpoints optimisés (index + filtres) et réponses JSON propres.

Sécurité

- Mots de passe hashés (Laravel).
- Auth par token (**Laravel Sanctum**).
- Autorisations : routes admin protégées (middleware + role check).

-
- Validation serveur sur toutes les entrées.
 - Protection contre injections SQL (Eloquent), XSS (escape côté React).

UI/UX

- Interface simple, claire, responsive.
- Feedback systématique : loaders, messages succès/erreur.

Compatibilité

- Chrome/Firefox/Safari/Edge.
- Responsive mobile (minimum).

Éco-responsabilité

- Pagination (éviter charges inutiles)
- Requêtes limitées (éviter refresh en boucle)
- Optimisation des bundles front (Vite, lazy loading si utile)

7. Architecture technique

Architecture globale

- **SPA React** (Vite) + **API REST Laravel**
- Données échangées en JSON

Front-end (React)

- React + React Router
- Gestion état : Context API ou Redux Toolkit (au choix)
- Client HTTP : Axios
- UI : Tailwind CSS (ou autre)

Back-end (Laravel)

- Laravel (API)
- DB : MySQL
- Auth : Laravel Sanctum (tokens)
- Structure API : routes / controllers / services (simple)

Communication

- React appelle l'API via Axios
- Token stocké côté client (localStorage ou cookie httpOnly selon choix)
 - **V1 simple : localStorage + header Authorization**

8. Interface utilisateur (écrans)

Visiteur

- Page Connexion
- Page Inscription

Membre

- Dashboard membre (statut cotisation du mois)
- Profil
- Mes cotisations (liste + filtre année)

Admin

- Dashboard admin (stats simples : total membres, payés ce mois)
- Liste membres (recherche + pagination)
- Détail membre (infos + paiements)
- Enregistrer un paiement (formulaire)

9. Flux de navigation (scénarios)

Scénario A – Inscription membre

1. Visiteur → "Inscription"
2. Remplit formulaire → Valider
3. API crée l'utilisateur (role=MEMBER, status=ACTIVE)
4. Redirection vers "Connexion" ou connexion auto

Scénario B – Connexion

1. Page Connexion → email/mdp
2. API renvoie token + user
3. Redirection :
 - ADMIN → Dashboard admin
 - MEMBER → Dashboard membre

Scénario C – Enregistrer une cotisation (admin)

1. Admin → Liste membres → choisir un membre
2. "Ajouter paiement" → saisir mois/année/montant/mode/date
3. API vérifie unicité (user+month+year)
4. Paiement enregistré → retour succès → historique mis à jour

Scénario D – Consultation membre

1. Membre → "Mes cotisations"
2. Filtre année
3. Affichage payé/non payé

10. Échéancier et plan de travail (1 semaine)

Jour 1 – Cadrage + Setup projet

- Création des projets :

-
- Backend : Laravel (API only)
 - Frontend : React + Vite
 - Configuration environnement :
 - Base de données (MySQL/PostgreSQL/SQLite)
 - Variables .env
 - CORS (API ↔ Front)
 - Initialisation Git + structure dossiers
 - Installation dépendances :
 - Laravel Sanctum
 - Axios (Front)
 - React Router

Jour 2 – Modèle de données + Auth API

- Migrations + modèles :
 - users (role, status)
 - contribution_payments (month, year, amount, method, date, unique)
- Seed / création d'un compte Admin par défaut
- Auth API :
 - Register (membre)
 - Login
 - Logout
- Validation back-end (Form Requests)

Jour 3 – API Admin (membres) + Sécurité

- Middleware auth + contrôle du rôle ADMIN
- Endpoints Admin :
 - Liste membres (pagination + recherche)
 - Détails membre + paiements
 - Activer / désactiver membre
- Gestion des erreurs (401/403/422) standardisées en JSON

Jour 4 – API Cotisations + Règles métier

-
- Endpoints Paiements :
 - Ajouter paiement (admin)
 - Historique global (admin, filtres mois/année/membre)
 - Historique personnel (membre)
 - Règles :
 - Unicité (user_id, month, year)
 - Contrôles montant, date, mode de paiement

Jour 5 – Front : Auth + Layout

- Pages :
 - Inscription
 - Connexion
- Gestion token + session (localStorage)
- Protected routes :
 - Routes admin
 - Routes membre
- Layout de base + navigation + déconnexion

Jour 6 – Front : Admin + Membre

- Admin :
 - Dashboard simple (stats)
 - Liste membres (recherche + pagination)
 - Détail membre + ajout paiement
- Membre :
 - Dashboard (statut cotisation mois en cours)
 - Mes cotisations (historique + filtre année)

Jour 7 – Tests, corrections et finalisation

- Tests manuels complets (scénarios)
- Ajustements UI/UX (messages, loaders, validations)
- Sécurisation finale (accès, erreurs, champs sensibles)
- README :

-
- Installation
 - Lancement API + Front
 - Compte admin (identifiants par défaut / création)
 - (Optionnel) mini démo + captures d'écran

11. Plan de validation et tests

Tests manuels (obligatoires)

- Inscription/connexion/déconnexion OK
- Accès interdit aux routes admin pour un membre
- Ajout paiement : refuse doublon (même mois/année)
- Pagination liste membres OK
- Statut payé/non payé affiché correctement

Tests automatisés (recommandés)

- Laravel Feature tests :
 - auth (register/login)
 - policy/middleware admin
 - création paiement + contrainte unique
- React :
 - tests simples composants (optionnel) ou tests E2E (optionnel)

12. Déploiement et exécution

Exécution locale

Prérequis :

- PHP + Composer
- Node.js + npm
- Base de données (MySQL)

Lancement :

- API : php artisan serve



Etudiant: SOULEYMANE MOUCTAR DIALLO

Cohorte: Développement Web Avancée.

-
- Front : npm run dev

13. Références

- Documentation Laravel (Sanctum, Eloquent, Validation)
- Documentation React, React Router, Axios